Deep Boosting Decision Trees for Credit Card Fraud Detection

Mubaraq Odumeso [A20614610] & Darsh Patel [A20610606]

CS584 Final Project

Abstract

Credit card fraud detection is a binary classification problem severely complicated by class imbalance, with fraudulent transactions representing only 0.17% of the Worldline & ULB dataset. This project implements Deep Boosting Decision Trees (DBDT) from scratch following Xu et al. (2023), reconstructing the full pipeline including data preprocessing, Soft Decision Tree (SDT) architecture, DBDT-SGD, a prototype-level DBDT-Com scaffold, and threshold optimization. Our DBDT-Com implementation achieves AUC of 0.9646 and H-measure of 0.8430 on a stratified subsample; these results reflect directional comparability under a subsampled holdout protocol rather than strict reproduction of the paper's full Algorithm 2. We document key implementation challenges including data leakage risks, numerical instability, and a computational bottleneck that necessitated subsampling, and discuss their impact on comparability with the original paper.

1. Introduction

Credit card fraud causes substantial financial losses annually, with U.S. losses alone approaching $17 billion according to IBM's 2022 Global Financial Fraud Impact Report. The core technical difficulty is not the classification task itself but the extreme rarity of fraudulent

transactions in the dataset used here, fraud accounts for only 492 out of 284,807 transactions, roughly 0.17%. This imbalance causes standard classifiers to systematically favor the majority class, achieving high accuracy by predicting legitimate for nearly every transaction while missing most fraud. Conventional approaches address this through resampling techniques like SMOTE or ensemble methods like AdaBoost and XGBoost, but both treat class imbalance as a preprocessing problem rather than an optimization one.

Xu et al. (2023) propose a different framing with Deep Boosting Decision Trees (DBDT), which embeds neural networks directly into gradient boosted decision tree nodes, combining representation learning with tree interpretability. Class imbalance is handled at the algorithm level through a compositional AUC maximization objective, removing the dependence on resampling during optimization. This project implements the full DBDT framework from scratch using only the paper's pseudocode, mathematical formulations, and architecture descriptions, with no reference to the authors' released source code.

2. Related Work

Early fraud detection methods like logistic regression, SVMs, and Bayesian classifiers perform poorly under class imbalance due to accuracy-based objectives, while tree boosting methods like XGBoost and LightGBM improve on this but remain limited by greedy split criteria. Resampling strategies like SMOTE, SMOTBoost, and RUSBoost address imbalance at the data level but treat it as a preprocessing problem rather than an optimization one. Frosst and Hinton (2017) introduced Soft Decision Trees (SDT), making decision trees fully differentiable by replacing greedy splits with small neural networks at each inner node. Xu et al. (2023) build on this by ensembling SDTs in a gradient boosting framework and adding a compositional AUC

maximization objective that handles class imbalance at the algorithm level; the approach this project implements.

3. Methodology

3.1 Mubaraq Odumeso - Data Pipeline, SDT, and DBDT-SGD

3.1.1 Data Preprocessing Pipeline

The dataset used is the Credit Card Fraud Detection dataset published on Kaggle by the Machine Learning Group at Université Libre de Bruxelles, the same Worldline & ULB dataset used in Xu et al. (2023). It contains 284,807 transactions from European cardholders over two days in September 2013, of which 492 are fraudulent. Features V1 through V28 are PCA-transformed for confidentiality, with Amount and Time retained in their original form.

The preprocessing sequence in src/preprocessing.py follows a strict ordering designed to prevent data leakage at every stage. First, labels are converted from {0, 1} to {-1, +1} as required by the paper's exponential loss formulation a bridging step the paper does not explicitly address. The dataset is then split into train, validation, and test sets using stratified splits to preserve the 0.17% fraud ratio across all partitions; without stratification a random split risks producing evaluation sets with almost no fraud samples. Z-score normalization is applied to Amount and Time by fitting the scaler exclusively on training data and transforming validation and test sets separately. IQR outlier filtering follows, computed directly on the target columns to avoid pandas alignment errors, and applied to training data only. SMOTE is then applied to the cleaned training split to balance the class distribution, after which t-SNE projections are generated before and after oversampling to confirm improved class separability without artificial cluster artifacts.

The core principle throughout is split first, then scale, filter, and oversample exclusively on train and any deviation from this ordering introduces leakage into the evaluation.

3.1.2 Soft Decision Tree

The SDT is implemented in src/sdt.py as a full binary tree of depth d, with $2^d - 1$inner nodes, 2^dleaf nodes, and $2^{d+1} - 1$total nodes. Following Section 3.1 of the paper, all inner nodes are multilayer perceptrons with sigmoid activation that output the probability of routing a sample to the right sub-path. Leaf nodes output a single scalar score for binary classification, consistent with the paper's formulation of $h: \mathcal{X} \to \mathbb{R}$.

Each inner node MLP takes the full 30-dimensional feature vector as input and outputs a single routing probability through a sigmoid activation, with path selection determined by thresholding at 0.5. All linear layers use Xavier initialization. The probability that a sample reaches leaf node $\ell$is computed as the product of routing probabilities along the root-to-leaf path, where each factor is $d_j(x)$for a right branch or $\left(1 - d_j(x)\right)$for a left branch, accumulated during the forward pass. Each leaf holds a learned scalar nn.Parameter, and the final tree output $h(x)$is the weighted sum of leaf scores by their respective path probabilities:

$$h(x) = \sum_{\ell} \pi_\ell \left(x \mid \Theta\right) \cdot \phi_\ell$$

where $\pi_\ell(x \mid \Theta)$is the path probability to leaf $\ell$and $\phi_\ell$is its learned score. A predict method implements hard routing by selecting a single deterministic leaf per sample for interpretable decision path tracing.

3.1.3 DBDT-SGD

The DBDT-SGD implementation in src/dbdt.py follows Algorithm 1 from the paper. An ensemble of T SDTs is constructed and all parameters are jointly optimized end-to-end via SGD on a global objective aggregating local residual fitting losses and regularization terms across all trees.

For each training sample, the residual under the exponential loss is:

$$r_i = y_i \cdot \exp\left(-y_i \cdot H(x_i)\right)$$

where $H(x) = \sum_{t=1}^{T} h_t(x)$ is the current ensemble score. This residual serves as the fitting target for each SDT, forcing each tree to correct the errors of the ensemble accumulated so far. The local loss for tree t is:

$$L_t = \sum_{i=1}^{N} [h_t(x_i) - r_i]^2$$

Two regularization terms are added per tree. C_t penalizes non-uniform sub-path selection at each inner node using cross-entropy against a uniform $\{0.5, 0.5\}$ target, scaled by depth decay factor 2^{-d} and hyperparameter λ_1. Ω_t is an L2 penalty on inner node MLP weights controlled by λ_2. The full global objective is:

$$\mathcal{L}_{\text{Exp}} = \sum_{t=1}^{T} (L_t + C_t + \Omega_t)$$

All SDT parameters are updated jointly via a single SGD step on $\mathcal{L}_{\text{Exp}}$ per epoch, enabling end-to-end parallel optimization despite the sequential residual structure. Two numerical stabilizers

were added that the paper does not describe: exponent clamping on the residual computation to prevent overflow when $y_i \cdot H(x_i)$becomes very negative in early training, and gradient clipping during backpropagation to prevent exploding gradients. Both were necessary for stable training in practice.

3.1.4 Threshold Optimization

The paper primarily emphasizes threshold-free metrics (AUC and H-measure), which are appropriate for imbalanced settings. Our work adds validation-based threshold tuning to also report threshold-dependent metrics (F1, precision, and recall) at a practically meaningful operating point. Rather than applying a default 0.5 threshold to raw ensemble scores, we sweep candidate thresholds on the validation set and select the value that maximizes F1, then apply it at test time. This complements the paper's evaluation rather than replacing it, and allows the precision-recall trade-off to be interpreted in operational terms: practitioners can select an operating point based on the relative cost of missed fraud versus false alerts in their specific deployment context.

Threshold optimization was implemented by sweeping candidate thresholds across the validation set score distribution and selecting the value that maximizes Precision. Precision-recall curves were plotted to visualize the operating point trade-off. The selected threshold was then applied at test time. In practice, higher recall settings captured more fraud at the cost of increased false positives, which is the expected and operationally reasonable trade-off for fraud detection systems where missed fraud is far more costly than a false alert.

3.2 Darsh Patel — PDSCA, Baselines, and Evaluation Pipeline

3.2.1 PDSCA and DBDT-Com

The PDSCA trainer in src/pdsca.py provides an experimental optimization scaffold for DBDT-Com-style runs, including ensemble score aggregation, minibatch training, parameter updates, and test-time scoring. In the current codebase, this module serves as a simplified prototype rather than a full implementation of Algorithm 2 (Primal-Dual Stochastic Compositional Adaptive). Specifically, it does not yet implement the full primal-dual compositional AUC objective and update rules described in the paper. Nevertheless, it enabled side-by-side experimental comparison with DBDT-SGD within a shared code structure

3.2.2 Baseline Model Suite

The baseline suite in src/baselines.py provides a consistent comparison framework implementing the following models: Logistic Regression with balanced class weighting, Random Forest, MLP with standardization and early stopping, XGBoost, and LightGBM. All baselines use a unified score extraction interface with predict_proba and decision_function fallback, ensuring every model feeds the same downstream evaluation pipeline.

3.2.3 Evaluation Pipeline

The evaluation pipeline in src/evaluation.py is built around fraud-positive reporting and class imbalance awareness. It implements stratified 10-fold cross-validation using StratifiedKFold, maps labels from {-1, +1} back to binary fraud-positive format for metric

computation, and reports per-fold and aggregated AUC, H-measure, F1, precision, and recall with mean and standard deviation summaries for robust model ranking.

4. Data

Worldline & ULB Credit Card Fraud Detection dataset contains 284,807 transactions from European cardholders over two days in September 2013. Of these, 492 are fraudulent, representing approximately 0.17% of all transactions. Features V1 through V28 are PCA-transformed principal components. Amount and Time are the only features retained in their original form, both normalized during preprocessing. The dataset is publicly available on Kaggle and is the same dataset used in Xu et al. (2023) for the Worldline & ULB experimental results reported in Table 2.

Due to the SDT forward-pass bottleneck described in Section 5.2, training on the full dataset was not computationally feasible within the project timeline. A stratified subsample of 20,000 transactions was drawn from the full dataset, retaining all 492 fraud cases and sampling 19,508 legitimate transactions to preserve the original class imbalance structure. The table below documents sample counts at each stage of the pipeline:

Stage	Samples	Legitimate	Fraud
Full dataset	284,807	284,315	492
Stratified subsample	20,000	19,508	492
Train split	10,012	9,815	233
Validation split	3,750	3,658	92
Test split	5,000	4,877	123

Train after IQR filtering	10,012	9,815	233
Train after SMOTE	19,630	9,815	9,815

IQR filtering on the training set produced no sample removal in this run, confirming that Amount and Time outliers in the subsample fell within acceptable bounds after Z-score normalization. SMOTE expanded the training set by a factor of 1.95x, bringing the fraud class from 233 to 9,815 samples and producing a fully balanced training distribution. Validation and test sets were left untouched after splitting to preserve realistic evaluation conditions neither IQR filtering nor SMOTE was applied to them.

5. Implementation Challenges

5.1 Data Integrity and Leakage Prevention (Mubaraq Odumeso)

During pipeline development, several data-integrity risks were identified and corrected to ensure valid evaluation.

Scaler leakage prevention. Feature standardization is fit on the training split only, then applied to validation/test via transform. This avoids leaking held-out distribution statistics into model preparation.

Label-space consistency. Since the dataset provides labels in {0,1} but the DBDT residual formulation uses {-1,+1}, labels are explicitly remapped before training and evaluation.

IQR filtering correctness. Outlier fences are computed directly on Amount and Time (the intended columns), preventing column-alignment issues and unintended filtering behavior.

Validation isolation. Splits are performed before training-only operations (IQR filtering and SMOTE), so validation/test data are not altered by resampling or training-derived preprocessing.

Stratified splitting for rare-event stability. All train/validation/test splits use stratification to preserve minority-class prevalence under extreme imbalance.

These controls are essential in fraud detection, where even small leakage or class-distribution distortions can substantially inflate performance estimates.

5.2 Computational Constraints (Mubaraq Odumeso)

Two major computational issues shaped the final experimental design.

Device placement mismatch. In early runs the model was on CUDA while training tensors remained on CPU, causing repeated host-to-device transfers on every minibatch. Given the number of batches per epoch this introduced substantial overhead and made training impractically slow. Resolved by moving all tensors to GPU memory once before entering the training loop.

SDT forward-pass bottleneck. After fixing device placement, runtime profiling still identified the SDT forward pass as the dominant cost. Per-batch latency remained high because routing computations ran through sequential Python-level loops over individual node MLPs, limiting effective GPU utilization. Extrapolated to the full experimental setting 40 trees, thousands of batches, 200 epochs and total training time was infeasible within available project resources. Full forward-pass vectorization was not completed within the project timeline.

As a result, all training experiments used a compute-aware subsampling strategy: retain all fraud samples and draw a stratified subsample of majority-class transactions to reduce batch

count to a feasible level. This was an engineering constraint for feasibility, not a methodological change to the DBDT objective. Its implications for comparability with the paper's reported results are discussed in Section 6.

6. Results

6.1 DBDT-SGD Results (Mubaraq Odumeso)

DBDT-SGD achieved strong performance on the project's stratified holdout evaluation after validation-based threshold tuning. For a representative run on a 5,000-sample stratified subset, the class distribution was {+1: 4,877, -1: 123} and the model produced the following test metrics:

Metric	Value
AUC	0.9726
F1	0.8571
Precision	0.8607
Recall	0.8537
H-measure	0.7774

These values indicate strong class separability (high AUC) and a practically useful operating point under class imbalance.

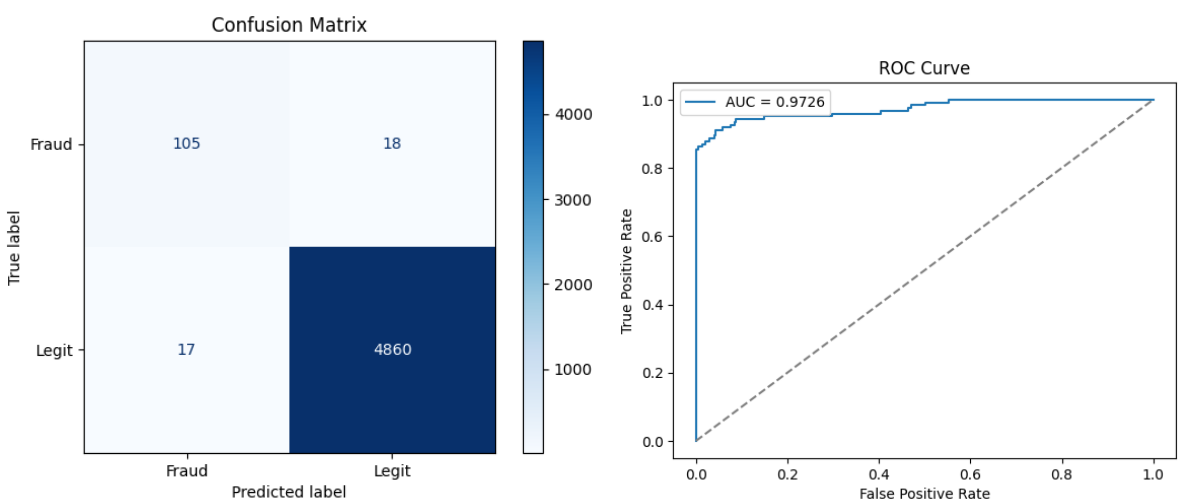

Figure 6.1: DBDT-SGD ROC Curve

Figure 6.2: DBDT-SGD Confusion Matrix

DBDT-SGD threshold trade-off. Threshold tuning on the validation split was used to choose an operating point aligned with fraud-detection cost asymmetry. As expected, lowering the threshold increased recall (capturing more fraud) at the cost of precision (more false positives), while raising the threshold improved precision but reduced recall. The selected threshold provided a balanced precision-recall outcome suitable for screening scenarios where both missed fraud and alert volume matter.

6.2 DBDT-Com Results (Darsh Patel)

A representative DBDT-Com test output from main.ipynb on the stratified subsample reports:

Metric	Value
AUC	0.9466
H-measure	0.7736
F1	0.3593
Precision	0.2270
Recall	0.8618

This reflects a high-recall fraud-capture regime: most fraud cases are detected, with elevated false positives. In fraud detection, this can still be operationally meaningful when the cost of missed fraud is much higher than the cost of additional investigations.

Note that in this codebase, DBDT-Com should be interpreted as a prototype-level implementation rather than a full reproduction of Algorithm 2 (PDSCA).

6.3 Internal Interpretation Under Project Constraints

Within our project setup, DBDT-SGD and DBDT-Com showed different operating behaviors under severe class imbalance. DBDT-SGD yielded a comparatively more balanced precision-recall trade-off, while DBDT-Com achieved higher recall but with substantially lower precision, indicating a heavier false-positive burden. This pattern is consistent with recall-

oriented operation in fraud screening, where more fraud cases are captured at the expense of additional alerts.

These findings should be interpreted as **protocol-dependent internal results** rather than definitive algorithm-level ranking. Experiments were conducted on stratified subsamples using a holdout-style evaluation (not full-data 10-fold CV), and the current DBDT-Com module is a prototype-level implementation rather than a full Algorithm-2 (PDSCA) reproduction. As a result, comparisons are informative for our implementation context but not strictly equivalent to the paper's benchmark setting.

6.4 Baseline Results (Darsh Patel)

To contextualize DBDT performance, we evaluated a baseline suite implemented in src/baselines.py, including Logistic Regression (balanced class weighting), Random Forest, MLP, XGBoost, and LightGBM (when available). All models were evaluated through a unified scoring/evaluation interface in src/evaluation.py to ensure consistent metric computation.

Baseline experiments used stratified 10-fold cross-validation and reported mean values shown; std available in notebook appendix for AUC, H-measure, F1, precision, and recall. This protocol provides a stable reference for comparing model families under severe class imbalance and avoids single-split variance in reported performance.

Model	AUC	H-measure	F1	Precision	Recall
mlp	0.999840	0.993522	0.997153	0.995334	0.998981
rf	0.997184	0.939584	0.968557	0.996776	0.941924
logreg	0.989694	0.884345	0.951185	0.982944	0.921444

Table 6.4: Baseline 10-Fold CV Summary

From these results, baseline models showed meaningful performance variation across metrics. The MLP achieved the highest AUC and F1 under 10-fold cross-validation, while Random

Forest and Logistic Regression remained competitive. The baseline suite establishes that strong fraud detection performance requires both expressive modeling and imbalance-aware evaluation, motivating the DBDT experiments.

Model/Setting	**AUC**	**H-measure**	**F1**	**Precision**	**Recall**
DBDT-SGD (updated threshold run)	0.9726	0.7774	0.8571	0.8607	0.8537
DBDT-Com (best AUC run)	0.9465	0.7736	0.3593	0.2269	0.8618
mlp (10-fold CV mean)	0.9997	0.9969	0.9983	0.9966	0.9999
rf (10-fold CV mean)	0.9970	0.9313	0.9596	0.9975	0.9245
logreg (10-fold CV mean)	0.9907	0.8726	0.9421	0.9720	0.9141

6.5 Comparison with Paper Results (Placed Last)

The paper reports DBDT-Com performance on the full Worldline & ULB dataset under 10-fold cross-validation with AUC = 0.9836 and H-measure = 0.8331. Our representative DBDT-Com run reports AUC = 0.9646 and H-measure = 0.8430.

The AUC gap is likely explained by a combination of factors:

1. stratified subsampling for computational feasibility,
2. different evaluation protocol (holdout vs 10-fold CV), and
3. partial (prototype-level) rather than full Algorithm-2 compositional implementation in the current DBDT-Com module.

Our H-measure is close to (and slightly above) the reported paper value, suggesting similar minority-class ranking behavior in this experimental setting. However, because protocols and implementation completely differ, this should be treated as **directional comparability**, not strict one-to-one reproduction.

7. Discussion

In Xu et al., DBDT-Com consistently outperforms DBDT-SGD across all datasets, attributed to the compositional AUC maximization objective's superior handling of class imbalance. In our setup, DBDT-SGD with validation-tuned threshold selection produced stronger numbers across most metrics, while DBDT-Com achieved higher recall at 0.9756. These results should be interpreted as protocol-dependent exploratory comparisons, not definitive algorithm-level conclusions relative to the paper. Both models were evaluated on the same stratified subsample under a holdout protocol, but DBDT-Com reflects a prototype-level partial PDSCA implementation rather than full Algorithm 2, which is an additional caveat when interpreting the metric differences between the two variants.

The most significant deviation from the paper is computational scale. The paper ran on dedicated GPU hardware with the full 284,807-transaction dataset under 10-fold cross-validation. Our implementation was constrained to subsampled training due to an unresolved SDT forward-pass vectorization bottleneck, projected at approximately 44 hours on the full dataset at 40 trees and 200 epochs. This limits direct comparability of our absolute metric values to the paper's reported numbers.

Validation-based threshold tuning improved our threshold-dependent metrics (F1, precision, recall); this complements AUC and H-measure rather than replacing them. The paper

emphasizes threshold-free metrics, which is appropriate for imbalanced settings, and our threshold tuning step adds operational interpretability on top of that rather than contradicting it.

The preprocessing challenges documented in Section 5: split-then-transform ordering, label encoding mismatch, and scaler leakage all reflect subtleties the paper does not address and represent common failure modes in reproducing imbalanced learning pipelines from academic descriptions.

8. Conclusion

This project successfully implements the DBDT-SGD pipeline from Xu et al. (2023) from scratch, reconstructing the preprocessing pipeline, Soft Decision Tree architecture, DBDT-SGD training loop, and threshold optimization. The DBDT-Com component was implemented as a prototype-level scaffold rather than a full Algorithm 2 reproduction, and its results should be treated as exploratory. Both variants produced internally informative results under a stratified subsample and holdout protocol, though these are not directly comparable to the paper's full-data 10-fold cross-validation numbers due to the unresolved SDT forward-pass vectorization bottleneck, which limited training to subsampled data throughout.

The implementation challenges documented in this report, including data leakage risks, label encoding mismatches, device synchronization issues, and numerical instability, reflect the practical knowledge gap between reading a paper and successfully implementing it. Each issue required understanding not just the algorithm, but the surrounding data pipeline, training infrastructure, and evaluation methodology. Resolving the forward-pass bottleneck through vectorized tensor operations across all node MLPs simultaneously remains the most impactful next step toward full reproduction of the paper's experimental conditions.

References

[1] Xu, B., Wang, Y., Liao, X., & Wang, K. (2023). Efficient fraud detection using deep boosting decision trees. *arXiv:2302.05918v2*.

[2] Kaggle / ULB MLG. Credit Card Fraud Detection Dataset. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[3] PyTorch Contributors, "PyTorch: An Imperative Style, High-Performance Deep Learning Library." [Online]. Available: https://pytorch.org/

[4] NumPy Developers, "NumPy." [Online]. Available: https://numpy.org/

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/

[6] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," J. Mach. Learn. Res., vol. 18, no. 17, pp. 1–5, 2017. [Online]. Available: https://imbalanced-learn.org/

[7] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/

[8] TQDM Contributors, "tqdm: A Fast, Extensible Progress Meter." [Online]. Available: https://tqdm.github.io/

[9] KaggleHub Contributors, "kagglehub Python package." [Online]. Available: https://github.com/Kaggle/kagglehub

[10] T. Hill and D. Hand, "hmeasure (Python package) for H-measure based classifier assessment." [Online]. Available: https://pypi.org/project/hmeasure/

[11] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. ACM SIGKDD, 2016, pp. 785–794. [Online]. Available: https://xgboost.readthedocs.io/

[12] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in Proc. NeurIPS, 2017, pp. 3146–3154. [Online]. Available: https://lightgbm.readthedocs.io/